

Mise en Cohérence des Objectifs de TIPE (MCOT) Optimisation de l'exécution des requêtes SQL itérées

Antoine Bertin—Paillette
2025-2026

Introduction

Mon camarade et moi sommes des utilisateurs de SQL ayant travaillé dans le passé avec des bases de données pour des projets web back-end. Aimant de plus comprendre et optimiser des programmes informatiques complexes, l'étude des moteurs des Systèmes de Gestion des Base de Données (SGBD) nous a paru évidente tant elle nous permettra l'étude de leur fonctionnement.

Positionnement thématique

Informatique Théorique, Informatique pratique

Mots-clefs

Mots-clefs – Base de donnée, SQL, Algèbre relationnelle

Keywords – Database, SQL, Relational algebra

Bibliographie commentée

Les bases de données représentent les fondations d'internet. Depuis l'invention des premiers moteurs pour le traitement des requêtes SQL, plusieurs paradigmes ont émergé. Le paradigme que nous avons considéré est les bases de données orientées colonne [1]. Il consiste à représenter les données groupées par colonne en opposition au modèle orienté ligne qui regroupe les lignes les unes à la suite des autres. Ce modèle permet une très grande efficacité de lecture sur les grandes bases

de données avec par exemple une réduction du temps de traitement de l'ordre de 1000 en moyenne en comparant deux SGBD l'un utilisant le paradigme orienté colonne [2, 1].

Pour avoir une lecture rapide, le choix du support (disque dur, SSD,...) de stockage des données est essentiel, un autre facteur est le nombre de données à lire, il est donc pertinent de s'intéresser à comment les arranger pour optimiser cette partie. Dans de nombreuses requêtes, nos données doivent vérifier un prédicat (c'est la partie après le SELECT en SQL), il est donc intéressant d'ordonner nos données en fonction de ce critère pour facilement lire uniquement les données utiles, cette idée permet d'introduire les B-Arbres [1]. Cependant le coût de calculer et de maintenir un B-arbre en temps réel est coûteux, il est donc parfois préférable de cracker nos colonnes en différents clusters de données au lieu de construire des données trop complexes. En général, le choix de comment ordonner peut être décidé en temps réel à l'aide de modèles statistiques [3].

Pour traiter une requête SQL, différentes étapes sont requises, il faut d'abord un parser pour transformer la chaîne de caractères que l'utilisateur envoie en un arbre syntaxique. Cet arbre est ensuite traduit en plan d'exécution par l'algebrizer. Il porte le nom de l'algèbre relationnelle, domaine massivement utilisé pour représenter les plans après les travaux de Codd. Celui-ci cherche en mémoire un plan similaire sauvegardé dans le cache ou le crée si il est inexistant [4, p. 348]. Ce plan est ensuite optimisé à travers différentes optimisations, il est ensuite exécuté [5] .

L'optimisation de plan se déroule, dans les modèles dit "volcano", basé sur système R [6], en deux étapes. D'abord il faut concevoir et optimiser le plan logique pour cela plusieurs optimisations ont été pensées et sont applicables dans de très nombreux cas [1]. Ces optimisations que l'on appellera statiques sont à différencier des autres optimisations qui dépendent d'estimations du résultat de certaine opération et de modèles statistiques [7]. Après avoir optimisé le plan logique, il faut optimiser le plan physique, c'est à dire choisir quel algorithme utiliser pour appliquer une opération [1], par exemple l'opération de jointure possède différents algorithmes dont au moins 4 sont implémenté avec notamment le Leap Frog Tree Join [8]. Pour se faire, différentes méthodes ont été imaginées, il est possible de choisir à l'avance quelles opérations seront choisies ou alors de concevoir un modèle d'estimation des coûts et de calculer le modèle ayant le plus faible coût [4, 6] via un parcours de graphe.

En Conclusion, nous avons évoqué deux aspects majeurs dans l'optimisation de la vitesse d'exécution des requêtes SQL. Premièrement, la manière dans laquelle sont ordonnées et lues nos données et dans un second temps, comment notre requête SQL va être traduite et exécutée.

Problématique retenue

Comment minimiser le temps d'exécution de requêtes SQL itérées dans le cadre de grands jeux de données ?

Objectifs du TIPE

J'ai décidé de me concentrer sur la représentation des données, les objectifs de mon TIPE étant :

1. Co-crédation d'un moteur SQL acceptant une grande partie de la syntaxe SQL de zéro en C++
2. Implémentation d'un paradigme de stockage colonne
3. Implémentation de stratégies d'optimisation réduisant le coût de lecture
4. Tests et mesures sur de grands jeux de données
5. Comparaison avec des systèmes existants (Sqlite, Postgres)

Références

- [1] Daniel ABADI et al. « **The Design and Implementation of Modern Column-Oriented Database Systems** ». In : *Foundations and Trends in Databases* 5 (2013).
- [2] Mike STONEBRAKER et al. « **C-store: a column-oriented DBMS** ». In : (août 2005).
- [3] Clément ROUVROY. « Hypothetical Index Benefit Estimation on Column-oriented Databases using Quantiles ». en.
- [4] Bailu DING, Vivek NARASAYYA et Surajit CHAUDHURI. « **Extensible Query Optimizers in Practice** ». en. In : *Foundations and Trends in Databases* 14.3-4 (déc. 2024).
- [5] Pierre SENELLART. « **Cours de Pierre Senellart** ». fr. In : (2017).
- [6] M M ASTRAHAN, BLASGEN et BLASGEN. « **System R** ». en. In : (juin 1976).

- [7] Hai LAN, Zhifeng BAO et Yuwei PENG. « **A Survey on Advancing the DBMS Query Optimizer: Cardinality Estimation, Cost Model, and Plan Enumeration** ». In : *Data Science and Engineering* 6.1 (mars 2021).
- [8] Todd L. VELDHUIZEN. « **Leapfrog Triejoin: A Worst-Case Optimal Join Algorithm** ». In : (déc. 2013).